



Unidad 6 - Chatbots y Sistemas de Diálogo

UNR - TUIA - Procesamiento de Lenguaje Natural

Docente teoría: Juan Pablo Manson - jpmanson@gmail.com - [LinkedIn](#)

1. Chatbots

Introducción

Los chatbots, asistentes virtuales o sistemas de diálogo son programas informáticos diseñados para simular una conversación con usuarios humanos, especialmente a través de Internet. Estos sistemas combinan tecnologías de inteligencia artificial como el procesamiento de lenguaje natural (NLP) y el aprendizaje automático (ML) para comprender, procesar y responder a las solicitudes verbales o textuales de los usuarios. A menudo se les asigna tareas específicas, como proporcionar información, asistencia al cliente, realizar reservas o controlar dispositivos y sistemas a través de comandos de voz o texto.

Aunque los términos "chatbots", "asistentes virtuales" y "sistemas de diálogo" a menudo se usan indistintamente, existen ciertas diferencias sutiles entre ellos basadas en sus funciones, capacidades y usos previstos. A continuación, se describen algunas diferencias entre estos:

1. Chatbots:

- **Función:** Diseñados principalmente para interactuar mediante texto con los usuarios, ofreciendo respuestas y realizando tareas simples basadas en un conjunto de entradas predefinidas o mediante la comprensión del lenguaje natural.
- **Capacidad:** Los chatbots varían desde sistemas basados en reglas muy básicos hasta sistemas más avanzados impulsados por inteligencia artificial.
- **Uso:** Comúnmente se implementan en sitios web y aplicaciones de mensajería para asistencia al cliente, FAQs, y para tareas específicas de comercio electrónico o servicios informativos o de soporte.

2. Asistentes Virtuales:

- **Función:** Estos sistemas están diseñados para comprender y ejecutar una amplia gama de tareas, como configurar alarmas, realizar llamadas, enviar mensajes, proporcionar información en tiempo real y controlar dispositivos inteligentes, a menudo a través de comandos de voz.
- **Capacidad:** Suelen incorporar capacidades más sofisticadas de inteligencia artificial, como el aprendizaje automático y la comprensión del contexto, lo que les permite realizar tareas más complejas y personalizadas.
- **Uso:** Son integrados en smartphones, altavoces inteligentes, y sistemas de automatización del hogar, siendo ejemplos prominentes Siri de Apple, Alexa de Amazon y el Asistente de Google.



Ambos alternativas se refieren a tecnologías que permiten la interacción entre humanos y máquinas, los chatbots tienden a ser más específicos y están centrados en tareas, los asistentes virtuales son más amplios y centrados en el usuario, pero podemos encontrar matices y otras herramientas como parte del sistema. En la actualidad se habla generalmente de chatbot, como una manera genérica de referirse a los sistemas conversacionales que utilizan IA para un entendimiento avanzado del lenguaje natural. (por ej: ChatGPT, Gemini, Claude, Grok, etc.)

Historia

La historia de los chatbots se remonta a mediados de la década de 1960, cuando ELIZA, uno de los primeros chatbots creados, fue inventado por Joseph Weizenbaum en el MIT. ELIZA fue diseñado para imitar las respuestas de un terapeuta con el fin de proporcionar apoyo terapéutico a los pacientes.

```
Welcome to

EEEEEE LL      IIII  ZZZZZZ  AAAAA
EE      LL      II    ZZ      AA  AA
EEEEEE LL      II    ZZZ      AAAAAA
EE      LL      II    ZZ      AA  AA
EEEEEE LLLLLL IIII ZZZZZZ  AA  AA

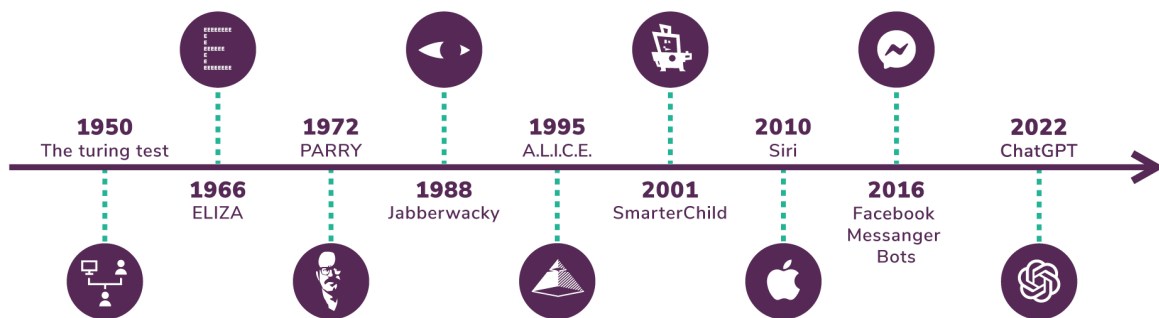
Eliza is a mock Rogerian psychotherapist.
The original program was described by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

ELIZA: Is something troubling you ?
YOU:   Men are all alike.
ELIZA: What is the connection, do you suppose ?
YOU:   They're always bugging us about something or other.
ELIZA: Can you think of a specific example ?
YOU:   Well, my boyfriend made me come here.
ELIZA: Is it important to you that your boyfriend made you come here ?
YOU:   He says I'm depressed much of the time.
ELIZA: I am sorry to hear that you are depressed.
YOU:   It's true. I am unhappy.
ELIZA: Can you explain what made you unhappy ?
YOU:
```

ELIZA fue diseñado para simular una conversación con un psicoterapeuta rogeriano mediante reglas sencillas de reconocimiento y sustitución de patrones, con las que respondía a las entradas del usuario. A pesar de su diseño rudimentario, quienes interactuaban con ELIZA a menudo sentían que conversaban con una persona que comprendía genuinamente sus emociones e inquietudes. Esa impresión no se debía a una comprensión avanzada ni a la empatía del programa, sino a la inteligencia que las personas le atribuían.

Por ejemplo, cuando un usuario escribía «Me siento triste», ELIZA podía responder: «¿Por qué te sientes triste?». Esta sencilla técnica de reflexión — que devolvía las palabras del usuario en forma de pregunta— lo animaba a seguir compartiendo pensamientos y sentimientos. Muchas personas comenzaron a revelar información personal y emocional, creyendo que ELIZA respondía con atención, aunque en realidad solo seguía patrones preestablecidos, sin comprender la situación.

En 1972, PARRY se convirtió en uno de los primeros chatbots diseñados para vencer a un oponente humano en una prueba de Turing. PARRY se basaba en las alucinaciones esquizofrénicas paranoicas de su creador, Kenneth Colby.



Desde entonces, los chatbots han recorrido un largo camino, evolucionando desde programas simples de coincidencia de patrones hasta su adopción masiva a principios de los años 2000 en servicios como AOL y MSN Messenger.

En 2016, la adopción generalizada de chatbots explotó cuando Facebook anunció que comenzaría a permitir bots en su popular plataforma de mensajería. Para el año 2018, había más de 300,000 chatbots activos en Facebook Messenger.

En los últimos años, la tecnología de los chatbots ha avanzado rápidamente debido a los avances en inteligencia artificial y procesamiento del lenguaje natural. Los chatbots de hoy son más sofisticados que nunca y se utilizan en una variedad de aplicaciones que van desde el servicio al cliente hasta el marketing y la atención médica.

La historia de los chatbots y los sistemas de diálogo ha experimentado varios puntos de inflexión significativos, especialmente con la aparición de los modelos de Transformers, que cambiaron la forma en que las máquinas entienden y generan lenguaje humano. Aquí hay un breve recuento desde la aparición de los Transformers hasta el desarrollo de ChatGPT:

2017: Aparición de los Transformers

- En junio de 2017, Google publica el artículo **"Attention Is All You Need"**, introduciendo la arquitectura Transformer. Esta nueva arquitectura basada en mecanismos de atención permitía a los modelos prestar atención a

diferentes partes de la entrada de texto de manera **más efectiva** que las técnicas anteriores como las redes neuronales recurrentes (**RNN**) y las redes de memoria a largo plazo (**LSTM**).

- Los Transformers marcaron el comienzo de una nueva era en el procesamiento del lenguaje natural (PLN), ofreciendo mejoras significativas en tareas como la traducción automática, el resumen de textos y la comprensión de lectura.

2018: BERT y GPT-2

- BERT (Bidirectional Encoder Representations from Transformers) fue presentado por Google en 2018 y demostró ser un punto de referencia en la capacidad de las máquinas para comprender el **contexto** en el texto.
- OpenAI lanzó GPT-2 en febrero de 2019. Aunque no fue inmediatamente liberado en su versión completa por temores de mal uso, GPT-2 fue un modelo de lenguaje que mejoró la generación de texto coherente y relevante.

2019-2020: GPT-3 y su impacto

- GPT-3 fue anunciado en junio de 2020 por OpenAI. Su modelo de 175 mil millones de parámetros superó significativamente a su predecesor en términos de tamaño y capacidad, generando texto que a menudo era indistinguible de lo escrito por humanos.
- GPT-3 fue ampliamente reconocido por su capacidad para realizar muchas tareas de NLP sin entrenamiento específico de la tarea, simplemente mediante instrucciones en lenguaje natural (prompts).

2021: Modelos de Transformers en aplicaciones prácticas

- Empresas y servicios empezaron a incorporar modelos como BERT y GPT-3 en sus productos, ofreciendo mejoras en chatbots, herramientas de escritura, motores de búsqueda y más.
- Los chatbots mejorados por estos modelos se hicieron más prevalentes en la atención al cliente, e-commerce y en aplicaciones de entretenimiento y educación.

2022-2023: ChatGPT y su evolución

- ChatGPT, basado en la arquitectura de GPT-3.5, fue optimizado específicamente para conversaciones, y lanzado por OpenAI a finales de

2022. Ofrece una experiencia de chat más fluida y contextualizada que las iteraciones anteriores.

- GPT-4: Mientras que GPT-3.5 solo acepta peticiones en texto, **GPT-4 es multimodal**, es decir, admite entradas en texto y visuales. O lo que es lo mismo: no tiene por qué ser una imagen con texto escrito (aunque valdría), si no que vale cualquier cosa, desde una fotografía de un paisaje a un problema matemático manuscrito pasando por un meme. GPT-4 es capaz de comprender y describir prácticamente cualquier imagen. Es decir, que GPT-4 ya no es "solo" un **modelo de lenguaje por Inteligencia Artificial**, **si no también un modelo visual**. Entre sus posibilidades está la de identificar objetos concretos dentro de una foto con muchos elementos visuales.

Clasificaciones

Los chatbots o sistemas de diálogo se han vuelto cada vez más populares debido a su capacidad para interactuar con los usuarios y ofrecer soluciones a diversas tareas. Pueden clasificarse según diversos criterios. Aquí vemos algunas de las clasificaciones más comunes:

1. Según la capacidad de procesamiento:

- **Chatbots basados en reglas:** Estos chatbots tienen respuestas predefinidas para preguntas específicas. Funcionan con un conjunto de reglas y no pueden manejar preguntas que no estén programadas en su sistema.
- **Chatbots basados en inteligencia artificial (IA):** Utilizan el procesamiento del lenguaje natural (PLN) y aprendizaje automático para entender y responder a las preguntas de los usuarios. Estos chatbots pueden aprender de las interacciones con los usuarios y mejorar con el tiempo.

2. Según su función:

- **Chatbots informativos:** Proporcionan información específica cuando se les pregunta, como el clima, noticias o datos sobre algún tema en particular.
- **Chatbots de servicio al cliente:** Están diseñados para resolver problemas específicos de los usuarios, como problemas con un producto o servicio.

- **Chatbots de ventas y marketing:** Ayudan a guiar a los usuarios a través de un proceso de compra o proporcionan recomendaciones de productos.
- **Chatbots en sitios web:** Están integrados en sitios web y asisten a los visitantes con tareas como navegación, preguntas frecuentes o soporte.
- **Chatbots de entretenimiento:** Son para entretener al usuario, como los chatbots que cuentan chistes o historias.

3. Según la plataforma:

- **Chatbots en aplicaciones de mensajería:** Operan dentro de aplicaciones de mensajería populares como WhatsApp, Facebook Messenger, Telegram, etc.
- **Chatbots en aplicaciones independientes:** Son aplicaciones móviles o de escritorio diseñadas específicamente para ofrecer funciones de chatbot.

4. Según el tipo de interacción:

- **Chatbots basados en texto:** Interactúan con los usuarios principalmente a través de texto.
- **Chatbots basados en voz:** Como los asistentes virtuales (ej. Alexa, Siri, Google Assistant), interactúan con los usuarios a través de comandos de voz.
- **Chatbots multimodales:** Combinan múltiples formas de interacción, como texto, voz, imágenes y más.

5. Según su diseño y personalidad:

- **Chatbots formales:** Están diseñados para ser directos y profesionales.
- **Chatbots amigables o casuales:** Tienen un tono más relajado y pueden usar emojis, GIFs y un lenguaje más coloquial.
- **Chatbots con personalidad definida:** Pueden estar diseñados para emular una personalidad específica, como un personaje famoso o un tipo particular de persona.

Chatbots Basado en Reglas

Como su nombre indica, estos chatbots utilizan una serie de reglas definidas. Con este tipo de bots, la comunicación se realiza a través de reglas predefinidas y un conjunto de preguntas.

Estos chatbots no son capaces de generar sus propias respuestas, pero con un extenso conjunto de respuestas y reglas inteligentemente diseñadas, pueden demostrar ser muy útiles y productivos.

También se les conoce como bots de árbol de decisiones y la razón es que los chatbots basados en reglas se guían por un árbol de decisiones, al cliente o usuario se le da un conjunto de opciones predefinidas que conducen a las respuestas deseadas.

Una limitación de estos chatbots es que no pueden responder preguntas fuera de las reglas definidas. Pero no se puede decir que sea una desventaja ya que el principal mecanismo de estos chatbots es responder preguntas limitadas por las reglas definidas.

Los chatbots basados en reglas se utilizan en escenarios donde un cliente o usuario desea realizar una tarea determinada, en ese caso, para automatizar ese proceso se prefieren los chatbots basados en reglas en lugar de los chatbots basados en IA. Esas tareas pueden ser, por ejemplo, reservar un pasaje de avión, reservar una entrada de cine, preguntar sobre horarios de vuelo y muchas más. La mayoría de las empresas utilizan chatbots basados en reglas para aumentar la satisfacción del cliente y responder a sus consultas.

Los chatbots basados en reglas se utilizan como recurso de preguntas frecuentes (FAQ) y no necesitan tener una gran cantidad de conversaciones de ejemplo para alimentar su respuesta.

Si bien estos chatbots son más “seguros” y “responsables”, las interacciones con el chatbot se sienten robóticas en lugar de conversacionales. Su rigidez puede producir un rechazo en algunos usuarios, ya que no se percibe una conversación natural.

Veamos un ejemplo básico:

```
import re
import random
from datetime import datetime

# Base de conocimiento del chatbot
knowledge_base = {
```



```

    'saludos': {
        'patrones': [
            r'\b(hola|hey|buenos días|buenas tardes|buenas
noches|hi|qué tal)\b'
        ],
        'respuestas': [
            '¡Hola! ¿Cómo puedo ayudarte?',
            '¡Bienvenido! ¿En qué puedo asistirte?',
            '¡Hola! Estoy aquí para ayudarte'
        ]
    },
    'despedidas': {
        'patrones': [
            r'\b(adiós|chau|hasta luego|nos vemos|bye)\b'
        ],
        'respuestas': [
            '¡Hasta luego! Que tengas un buen día',
            '¡Adiós! Gracias por tu visita',
            '¡Nos vemos pronto!'
        ]
    },
    'agradecimientos': {
        'patrones': [
            r'\b(gracias|te agradezco|muchas gracias)\b'
        ],
        'respuestas': [
            '¡De nada! Estoy para ayudar',
            'Es un placer poder ayudarte',
            'No hay de qué. ¿Necesitas algo más?'
        ]
    },
    'preguntas_estado': {
        'patrones': [
            r'\b(cómo estás|qué tal estás|cómo te encuentra
s)\b'
        ],
        'respuestas': [
            'Muy bien, ¡gracias por preguntar! ¿En qué pued

```

```

o ayudarte?',
        'Excelente y listo para ayudarte. ¿Qué necesita
s?',
        '¡Todo bien! ¿Cómo puedo asistirte hoy?'
    ]
},
'productos': {
    'patrones': [
        r'\b(qué productos|qué vendés|qué tienen|qué of
recen)\b'
    ],
    'respuestas': [
        'Tenemos una amplia variedad de productos: lapt
ops, smartphones, tablets y accesorios',
        'Ofrecemos productos electrónicos como computad
oras, celulares y tablets',
        'Vendemos todo tipo de dispositivos electrónico
s y sus accesorios'
    ]
},
'precios': {
    'patrones': [
        r'\b(cuánto cuesta|precio|valor|cuánto vale)\b'
    ],
    'respuestas': [
        'Los precios varían según el producto específic
o. ¿Qué producto te interesa?',
        'Para darte el precio exacto, ¿podrías decirme
qué producto te interesa?',
        '¿Qué producto específico te gustaría consulta
r?'
    ]
}
}

# Productos y sus precios
productos = {
    'laptop': {'precio': 800, 'descripcion': 'Laptop de últ

```

```

ima generaci3n'},
    'smartphone': {'precio': 500, 'descripcion': 'Smartphon
e con gran c3mara'},
    'tablet': {'precio': 300, 'descripcion': 'Tablet perfec
ta para entretenimiento'},
    'auriculares': {'precio': 50, 'descripcion': 'Auricular
es inal3mbricos'},
}

def get_time_based_greeting():
    """Retorna un saludo basado en la hora del d3a"""
    hora = datetime.now().hour
    if 5 <= hora < 12:
        return "¡Buenos d3as!"
    elif 12 <= hora < 20:
        return "¡Buenas tardes!"
    else:
        return "¡Buenas noches!"

def buscar_producto(mensaje):
    """Busca menciones de productos en el mensaje"""
    for producto in productos.keys():
        if producto in mensaje.lower():
            return producto
    return None

def procesar_mensaje(mensaje):
    """Procesa el mensaje del usuario y retorna una respues
ta apropiada"""
    mensaje = mensaje.lower()

    # Buscar producto espec3fico
    producto = buscar_producto(mensaje)
    if producto:
        if 'precio' in mensaje or 'cuesta' in mensaje or 'v
alor' in mensaje:
            return f"El {producto} cuesta ${productos[produ
cto]['precio']}. {productos[producto]['descripcion']}"

```

```

        return f"Tenemos {producto}. {productos[producto]
['descripcion']}. ¿Te gustaría saber el precio?"

# Buscar coincidencias en la base de conocimiento
for categoria, datos in knowledge_base.items():
    for patron in datos['patrones']:
        if re.search(patron, mensaje):
            return random.choice(datos['respuestas'])

# Si no se encuentra coincidencia
return "No estoy seguro de entender. ¿Podrías reformular tu pregunta?"

def main():
    """Función principal del chatbot"""
    print(f"{get_time_based_greeting()} Soy tu asistente virtual.")
    print("Puedo ayudarte con información sobre nuestros productos y precios.")
    print("Escribe 'salir' para terminar la conversación.\n")

    historial = []

    while True:
        try:
            # Obtener input del usuario
            entrada_usuario = input("Tú: ").strip()

            # Guardar en historial
            historial.append(("usuario", entrada_usuario))

            # Verificar si el usuario quiere salir
            if entrada_usuario.lower() == 'salir':
                print("Chatbot: ¡Gracias por tu visita! ¡Ha sta pronto!")
                break

```

```

        # Procesar el mensaje y obtener respuesta
        respuesta = procesar_mensaje(entrada_usuario)
        print("Chatbot:", respuesta)

    # Guardar respuesta en historial
    historial.append(("bot", respuesta))

except KeyboardInterrupt:
    print("\nChatbot: Programa terminado por el usu
ario.")
    break
except Exception as e:
    print(f"Chatbot: Lo siento, ha ocurrido un erro
r. Por favor, intenta de nuevo.")
    continue

if __name__ == "__main__":
    main()

```

Y la salida podría ser:

```

¡Buenas noches! Soy tu asistente virtual.
Puedo ayudarte con información sobre nuestros productos y precios.
Escribe 'salir' para terminar la conversación.

Tú: hola
Chatbot: ¡Bienvenido! ¿En qué puedo asistirte?
Tú: quiero saber precio de una tablet
Chatbot: El tablet cuesta $300. Tablet perfecta para entretenimiento
Tú: y la laptop?
Chatbot: Tenemos laptop. Laptop de última generación. ¿Te gustaría saber el precio?
Tú: si
Chatbot: No estoy seguro de entender. ¿Podrías reformular tu pregunta?
Tú: quiero precio de la laptop
Chatbot: El laptop cuesta $800. Laptop de última generación

Chatbot: Programa terminado por el usuario.

```

Al probar nuestro chatbot, notaremos las limitaciones de nuestra implementación, y las pocas posibilidades de conversación. Podríamos intentar otra estrategia, usando algunas capacidades de la librería `Spacy`:

```

# !pip install spacy
# !python -m spacy download es_core_news_md

import spacy
import random
from datetime import datetime
import re

# Cargar el modelo preentrenado de spaCy
nlp = spacy.load('es_core_news_md')

# Información ampliada sobre productos y promociones
productos = {
    'lavarropas': {
        'precio': 300,
        'descuento': 10,
        'características': ['6kg de capacidad', 'Eficiencia
energética A++', 'Multiple programas de lavado'],
        'stock': 5
    },
    'aspiradora': {
        'precio': 200,
        'descuento': 5,
        'características': ['Sin bolsa', 'Filtro HEPA', 'Po
tencia 2000W'],
        'stock': 8
    },
    'heladera': {
        'precio': 1000,
        'descuento': 20,
        'características': ['No frost', '500L de capacida
d', 'Dispensador de agua'],
        'stock': 3
    },
    'microondas': {
        'precio': 150,
        'descuento': 15,
        'características': ['30L de capacidad', 'Grill inte

```

```

grado', 'Panel digital'],
    'stock': 10
},
'licuadora': {
    'precio': 50,
    'descuento': 5,
    'características': ['Vaso de vidrio', '5 velocidades', '600W de potencia'],
    'stock': 15
},
'cafetera': {
    'precio': 40,
    'descuento': 5,
    'características': ['Capacidad 1.5L', 'Sistema anti goteo', 'Filtro permanente'],
    'stock': 12
}
}

```

Promociones con fechas de validez

```

promociones = [
    {
        "mensaje": "¡Este fin de semana todos los productos con 15% de descuento!",
        "validez": "hasta el domingo",
        "descuento": 15,
        "productos_aplicables": "todos"
    },
    {
        "mensaje": "Por la compra de una heladera, lleva una licuadora con 50% de descuento",
        "validez": "próximos 30 días",
        "descuento": 50,
        "productos_aplicables": ["heladera", "licuadora"]
    },
    {
        "mensaje": "Compra una lavadora y secadora en combo y ahorra $100",

```



```

        "validez": "stock disponible",
        "descuento_fijo": 100,
        "productos_aplicables": ["lavarropas", "secadora"]
    }
]

# Palabras clave para diferentes intenciones
intenciones = {
    'precio': ['precio', 'costo', 'vale', 'valor', 'cuánto'],
    'caracteristicas': ['características', 'especificaciones', 'detalles', 'funciones'],
    'stock': ['stock', 'disponible', 'hay', 'tienen'],
    'promocion': ['promoción', 'descuento', 'oferta', 'rebaja'],
    'comparacion': ['mejor', 'diferencia', 'comparar', 'versus', 'vs']
}

def detectar_intencion(message):
    """Detecta la intención principal del mensaje del usuario"""
    message = message.lower()
    intenciones_detectadas = []

    for intencion, palabras_clave in intenciones.items():
        if any(palabra in message for palabra in palabras_clave):
            intenciones_detectadas.append(intencion)

    return intenciones_detectadas if intenciones_detectadas else ['general']

def calcular_precio_final(producto):
    """Calcula el precio final con descuento"""
    precio_base = productos[producto]['precio']
    descuento = productos[producto]['descuento']
    precio_final = precio_base * (1 - descuento/100)

```

```

    return precio_final

def buscar_producto(message):
    """Busca productos mencionados en el mensaje usando lematización"""
    doc = nlp(message.lower())
    lemas = [token.lemma_ for token in doc]
    productos_encontrados = []

    for producto in productos:
        if producto in lemas:
            productos_encontrados.append(producto)

    return productos_encontrados

def generar_respuesta_producto(producto, intenciones):
    """Genera una respuesta detallada según el producto y las intenciones detectadas"""
    respuesta = []
    datos = productos[producto]

    if 'precio' in intenciones:
        precio_final = calcular_precio_final(producto)
        respuesta.append(f"El {producto} tiene un precio de ${datos['precio']}, con un {datos['descuento']}% de descuento. "
                        f"Precio final: ${precio_final:.2f}")

    if 'características' in intenciones:
        respuesta.append(f"Características principales: {' '.join(datos['características'])}")

    if 'stock' in intenciones:
        respuesta.append(f"Actualmente tenemos {datos['stock']} unidades disponibles")

    if 'general' in intenciones:

```

```

        precio_final = calcular_precio_final(producto)
        respuesta.append(f"El {producto} está disponible por ${precio_final:.2f} (incluye {datos['descuento']}% de descuento). "
                        f"Tenemos {datos['stock']} unidades en stock.")

    return " ".join(respuesta)

def comparar_productos(productos_lista):
    """Compara dos o más productos"""
    if len(productos_lista) < 2:
        return "Por favor, menciona al menos dos productos para comparar."

    comparacion = "Comparación de productos:\n\n"
    for producto in productos_lista:
        datos = productos[producto]
        precio_final = calcular_precio_final(producto)
        comparacion += f"{producto.capitalize()}: \n"
        comparacion += f"- Precio final: ${precio_final:.2f}\n"
        comparacion += f"- Características: {' '.join(datos['características'])}\n"
        comparacion += f"- Stock disponible: {datos['stock']}\n\n"

    return comparacion

def process_message(message):
    """Procesa el mensaje del usuario y genera una respuesta apropiada"""
    # Detectar intenciones
    intenciones_detectadas = detectar_intencion(message)

    # Buscar productos mencionados
    productos_encontrados = buscar_producto(message)

```

```

# Si no se encontraron productos
if not productos_encontrados:
    if 'promocion' in intenciones_detectadas:
        promo = random.choice(promociones)
        return f"¡Tenemos esta promoción especial: {promo['mensaje']} ({promo['validez']})"
    return "No he identificado ningún producto específico. ¿Puedes ser más específico sobre qué producto te interesa?"

# Si se encontró más de un producto y se detecta intención de comparación
if len(productos_encontrados) > 1 and ('comparacion' in intenciones_detectadas):
    return comparar_productos(productos_encontrados)

# Generar respuesta para cada producto encontrado
respuestas = []
for producto in productos_encontrados:
    respuesta = generar_respuesta_producto(producto, intenciones_detectadas)
    respuestas.append(respuesta)

return "\n".join(respuestas)

def main():
    """Función principal para ejecutar el chatbot"""
    print("¡Hola! Soy el chatbot de ElectroMax. Puedo ayudarte con:")
    print("- Información de productos y precios")
    print("- Características y especificaciones")
    print("- Disponibilidad y stock")
    print("- Promociones vigentes")
    print("- Comparación entre productos")
    print("\nEscribe 'salir' para terminar la conversación.")

    while True:

```

```

try:
    user_input = input("\nTú: ")
    if user_input.lower() == 'salir':
        print("Chatbot: ¡Gracias por visitar Electr
oMax! ¡Hasta pronto!")
        break

    bot_response = process_message(user_input)
    print("\nChatbot:", bot_response)

except KeyboardInterrupt:
    print("\nChatbot: Sesión terminada por el usuar
io. ¡Hasta pronto!")
    break
except Exception as e:
    print(f"\nChatbot: Lo siento, ha ocurrido un er
ror. Por favor, intenta de nuevo.")
    continue

if __name__ == "__main__":
    main()

```

Y la conversación podría ser:

¡Hola! Soy el chatbot de ElectroMax. Puedo ayudarte con:

- Información de productos y precios
- Características y especificaciones
- Disponibilidad y stock
- Promociones vigentes
- Comparación entre productos

Escribe 'salir' para terminar la conversación.

Tú: ¿Qué características tienen el microondas y la aspiradora?

Chatbot: Características principales: Sin bolsa, Filtro HEPA, Potencia 2000W Actualmente tenemos 8 unidades disponibles

Características principales: 30L de capacidad, Grill integrado, Panel digital Actualmente tenemos 10 unidades disponibles

Tú: ¿Cuál es el precio final del microondas con descuento?

Chatbot: El microondas tiene un precio de \$150, con un 15% de descuento. Precio final: \$127.50

Tú: ¿Qué funciones tiene el microondas?

Chatbot: Características principales: 30L de capacidad, Grill integrado, Panel digital

Tú: ¿Tienen heladera en stock?

Chatbot: Actualmente tenemos 3 unidades disponibles

Tú: ¿Tienen ofertas este fin de semana?

Chatbot: ¡Tenemos esta promoción especial: Por la compra de una heladera, lleva una licuadora con 50% de descuento (próximos 30 días)

Tú: ¿Qué diferencia hay entre el microondas y la cafetera?

Chatbot: Actualmente tenemos 10 unidades disponibles

Tú: ¿Qué características tiene la heladera y cuánto cuesta?

Chatbot: El heladera tiene un precio de \$1000, con un 20% de descuento. Precio final: \$800.00 Características principales: No frost, 500L de capacidad, Dispensador de agua

Tú: ¿Me podés decir los precios de la licuadora y la cafetera?

Chatbot: El licuadora tiene un precio de \$50, con un 5% de descuento. Precio final: \$47.50

Tú: ¿Hay stock de heladera y lavarropas?

Chatbot: Actualmente tenemos 3 unidades disponibles

Tú: ¿Cuáles son los electrodomésticos más vendidos?

Chatbot: No he identificado ningún producto específico. ¿Puedes ser más específico sobre qué producto te interesa?

Tú: ¿Tienen servicio de entrega?

Chatbot: No he identificado ningún producto específico. ¿Puedes ser más específico sobre qué producto te interesa?

Tú: Hola, estoy buscando una heladera nueva, ¿qué me recomendás?

Chatbot: El heladera está disponible por \$800.00 (incluye 20% de descuento). Tenemos 3 unidades en stock.

Chatbot: Sesión terminada por el usuario. ¡Hasta pronto!

La primera versión del chatbot que vimos anteriormente, utilizaba un enfoque basado en expresiones regulares. Las expresiones regulares son patrones que permiten buscar coincidencias dentro de cadenas de texto. Esto significa que

son muy potentes para buscar patrones específicos, pero tienen algunas limitaciones:

1. **Literalidad:** Las expresiones regulares buscan coincidencias exactas o patrones muy específicos. Si un usuario no escribe exactamente lo que la expresión regular está configurada para encontrar, es posible que el chatbot no reconozca la intención.
2. **Flexibilidad:** Las expresiones regulares no entienden el lenguaje natural; solo siguen las reglas específicas del patrón. Esto significa que no manejan bien las variaciones lingüísticas como las conjugaciones de verbos, los plurales, las palabras compuestas, etc.
3. **Mantenimiento:** A medida que se agregan más funcionalidades y se cubren más variaciones de lenguaje natural, las expresiones regulares se pueden volver muy complejas y difíciles de mantener.

El chatbot mejorado con spaCy ofrece varias mejoras:

1. **Lematización:** Al reducir las palabras a su raíz o forma base (lemas), el bot puede reconocer "aspirar", "aspirando", "aspirado" como relacionados con la palabra "aspiradora". Esto le permite manejar variaciones lingüísticas mucho más naturalmente que las expresiones regulares.
2. **Comprensión de lenguaje natural:** spaCy ofrece un análisis más sofisticado del texto que incluye la identificación de entidades nombradas (NER), categorización de palabras (part-of-speech tagging), y dependencias sintácticas, lo que significa que el chatbot puede tener una comprensión más rica del texto y la intención del usuario.
3. **Escalabilidad y mantenimiento:** Al depender del procesamiento del lenguaje natural (NLP), es más fácil escalar y mantener el chatbot porque se basa en modelos lingüísticos en lugar de una lista siempre creciente de patrones de expresiones regulares.
4. **Contextualización:** Con spaCy, y en general con cualquier librería de NLP, hay un potencial para utilizar el contexto del diálogo anterior para informar respuestas futuras, algo que las expresiones regulares no pueden hacer directamente.

Chatbots basados en IA

Los chatbots basados en IA, específicamente los que utilizan Modelos de Lenguaje a Gran Escala (LLMs, por sus siglas en inglés), son sistemas

avanzados de procesamiento de lenguaje natural que se basan en redes neuronales profundas. Estos son algunos de los puntos clave que definen cómo funcionan y se aplican:

Tecnología de Aprendizaje Profundo:

- Los LLMs utilizan arquitecturas de aprendizaje profundo como las redes neuronales recurrentes (RNN), las redes neuronales convolucionales (CNN) y, más recientemente, las redes de atención y transformadores.
- Estas redes están diseñadas para procesar secuencias de palabras y entender la estructura y el contexto del lenguaje.

Entrenamiento con Grandes Conjuntos de Datos:

- Se entrenan con enormes corpus de texto que pueden abarcar desde enciclopedias y literatura hasta diálogos y foros en línea.
- Aprenden a predecir la siguiente palabra en una secuencia, lo que les ayuda a generar respuestas coherentes y contextualmente relevantes.

Capacidad de Generalización:

- A diferencia de los sistemas basados en reglas, los LLMs pueden generalizar a partir de los datos de entrenamiento y manejar preguntas o comandos que nunca han visto antes.
- Esto los hace mucho más flexibles y capaces de mantener conversaciones más fluidas y naturales.

Comprensión y Generación de Lenguaje Natural:

- Los LLMs utilizan procesamiento de lenguaje natural (NLP) para analizar y entender las consultas de los usuarios.
- Pueden manejar ambigüedades, errores comunes en el lenguaje humano y entender diferentes matices, como el sarcasmo o el humor, dependiendo de su entrenamiento y sofisticación.

Aprendizaje Continuo:

- Algunos LLMs pueden mejorar con el tiempo a través del aprendizaje continuo, adaptándose a los patrones de habla de los usuarios y refinando sus modelos internos para ofrecer respuestas más precisas.

Aplicaciones de LLMs:

- Asistencia virtual: Pueden realizar tareas como establecer recordatorios, responder preguntas y ayudar con las tareas diarias.
- Atención al cliente: Proporcionan respuestas automáticas a preguntas frecuentes, resolución de problemas y asistencia en transacciones en línea.
- Terapia y asesoramiento: Algunos chatbots están diseñados para ofrecer apoyo emocional y consejos, aunque con ciertas limitaciones y siempre bajo supervisión humana.

Desafíos y Consideraciones:

- Sesgo y ética: Los LLMs pueden perpetuar sesgos presentes en los datos de entrenamiento, lo que requiere cuidadosa atención y ajuste.
- Privacidad y seguridad: Manejan información sensible del usuario, lo que exige protocolos robustos de seguridad y privacidad.
- Limitaciones contextuales: A pesar de su sofisticación, pueden malinterpretar el contexto o la intención, llevando a respuestas inadecuadas o irrelevantes.

Capacidades emergentes de los LLM

Las **capacidades emergentes de un LLM (Large Language Model)** son habilidades que no están explícitamente programadas pero que aparecen cuando el modelo alcanza cierto tamaño o se entrena con suficiente diversidad de datos. A continuación, se describen algunas de ellas, junto con una explicación de por qué ocurren:

Razonamiento aritmético y lógico básico

A medida que el modelo crece, puede resolver problemas matemáticos sencillos o inferencias lógicas, aunque no haya sido diseñado para ello. Esto ocurre porque el entrenamiento sobre grandes cantidades de texto incluye ejemplos de razonamiento numérico y verbal, y el modelo aprende patrones estadísticos que generalizan a nuevas preguntas. La arquitectura autoatencional también ayuda a seguir relaciones entre tokens que representan pasos lógicos.

Traducción entre lenguas

Incluso sin supervisión directa, un LLM puede traducir textos entre idiomas si ha visto suficiente texto multilingüe. Esto se debe a que aprende una

representación semántica compartida: identifica que ciertas estructuras tienen significados equivalentes, aunque se expresen con diferentes palabras. Esta capacidad emerge por el alineamiento estadístico de significados similares en múltiples lenguas

Seguimiento de instrucciones complejas

Los LLM grandes muestran una capacidad sorprendente para seguir instrucciones dadas en lenguaje natural, incluso si son detalladas o de múltiples pasos. Esto emerge porque durante el entrenamiento, el modelo ve muchos ejemplos de diálogos, manuales y patrones de pregunta-respuesta. El tamaño permite que generalice estas estructuras y represente contextos amplios de manera coherente.

Comprensión contextual de metas y roles

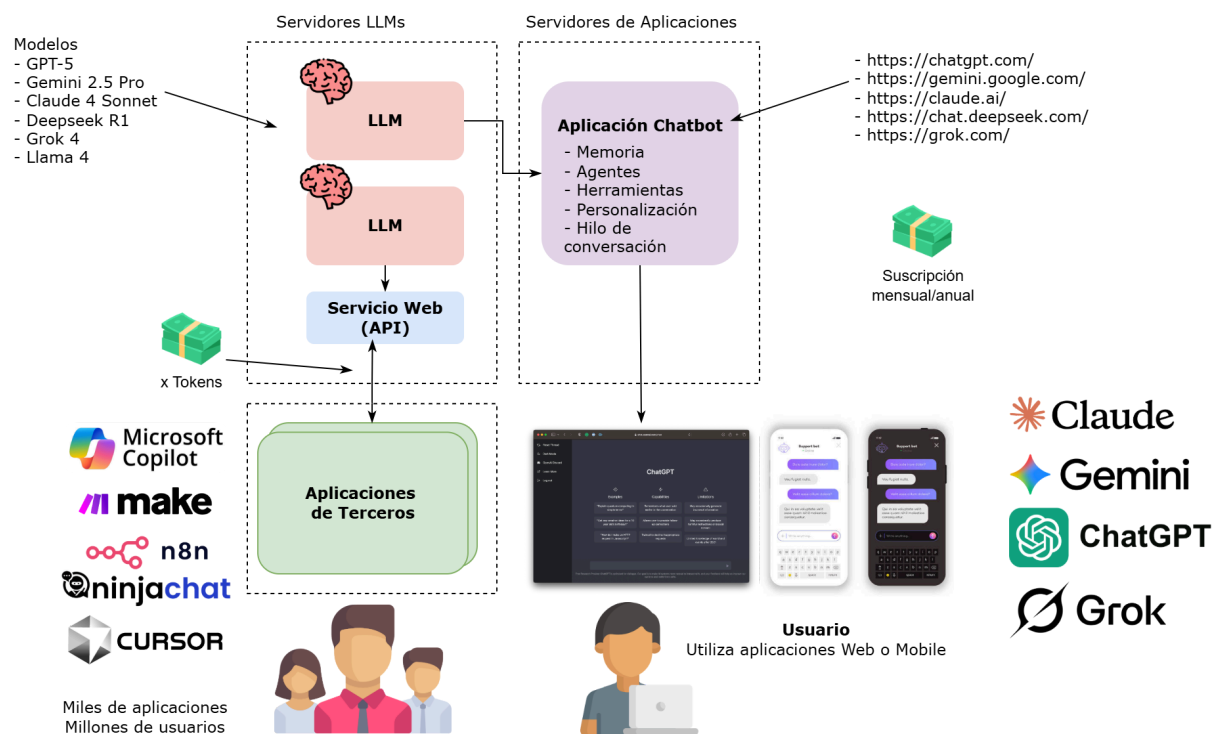
En interacciones complejas, los LLM pueden adoptar roles (como médico, abogado o tutor) y responder en consecuencia. Esto ocurre porque el modelo ha visto en el entrenamiento numerosos ejemplos de interacciones donde se especifica un rol, y aprende a asociar estilos de lenguaje, objetivos y convenciones de cada uno. La capacidad de mantener coherencia de rol es emergente y depende del tamaño del modelo y de la riqueza del corpus.

Aplicaciones basadas en LLM

El siguiente esquema representa el ecosistema tecnológico que hace posible la interacción con chatbots basados en modelos de lenguaje (LLM). En el centro del diagrama se observa que los LLM funcionan como el "cerebro" de la inteligencia artificial, capaces de procesar lenguaje natural, razonar y generar respuestas. Estos modelos no interactúan directamente con los usuarios, sino que lo hacen a través de un Servicio Web (API), que actúa como intermediario y permite que otras aplicaciones puedan solicitar respuestas o ejecutar tareas basadas en inteligencia artificial.

Sobre esta infraestructura se construye la aplicación chatbot, que agrega funcionalidades avanzadas como memoria, personalización, manejo del hilo conversacional, agentes y herramientas. Esta capa es la responsable de brindar una experiencia inteligente, adaptativa y contextualizada. Finalmente, los usuarios acceden a estos servicios mediante distintas interfaces: aplicaciones web o móviles, plataformas comerciales como ChatGPT, Microsoft Copilot o herramientas integradas como n8n, Make o Cursor. De este modo, el gráfico muestra cómo los LLM, a través de una arquitectura modular y

conectable por API, se integran con múltiples aplicaciones y llegan al usuario final como asistentes conversacionales personalizados y útiles.



Ejemplos prácticos

Google Colab

 <https://colab.research.google.com/drive/1UQqSRUI4mflfwuQfcogpHueSftFKIXsY?usp=sharing>



2. Prompts

En el contexto de la Procesamiento del Lenguaje Natural (NLP) y los Modelos de Lenguaje a Gran Escala (LLM, por sus siglas en inglés), un "prompt" se refiere a una instrucción o conjunto de instrucciones dadas a un modelo de lenguaje para que realice una tarea específica. Los prompts actúan como una interfaz entre el usuario y el modelo, indicándole al modelo qué se espera de él.

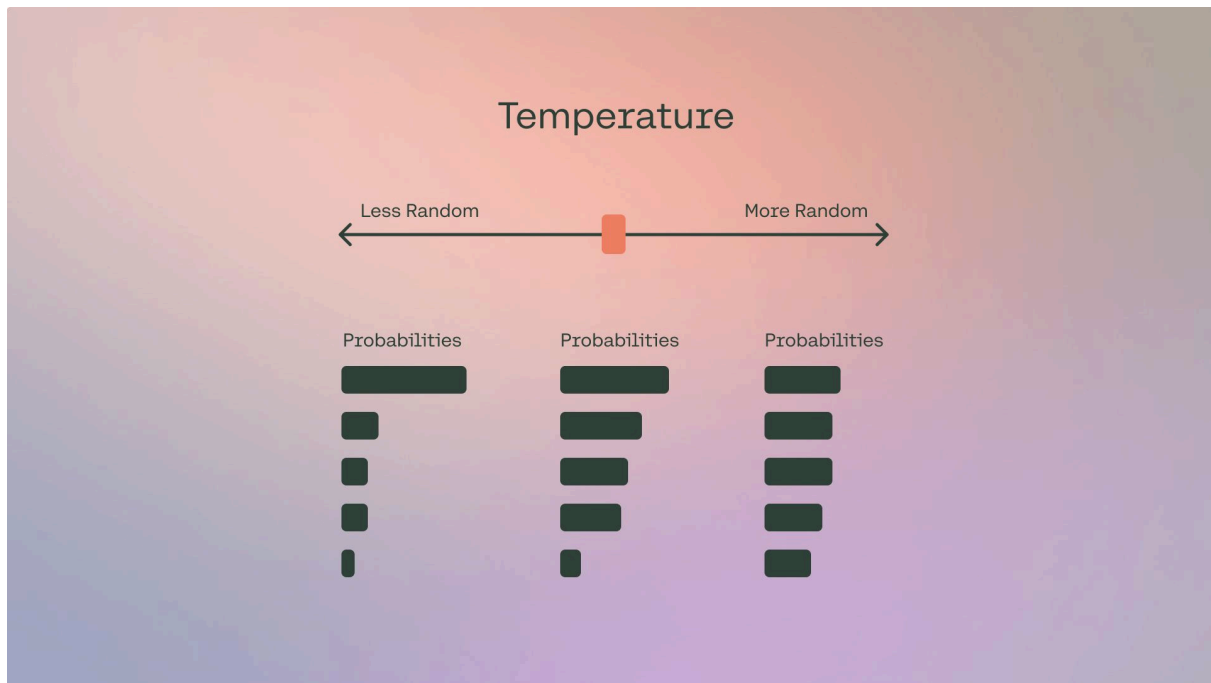
Por ejemplo, en la generación de texto, un prompt podría ser una frase o un párrafo que el modelo usa como punto de partida para generar texto continuo. En el caso de las preguntas y respuestas, el prompt sería la pregunta formulada al modelo, a la que se espera que responda.

Los prompts son fundamentales en la interacción con los LLMs porque los modelos están diseñados para responder o seguir las instrucciones contenidas en los prompts. La calidad y claridad de un prompt pueden influir significativamente en la calidad de la salida del modelo. Por esta razón, el diseño de prompts efectivos es una habilidad importante en el campo de NLP, ya que permite a los usuarios aprovechar al máximo las capacidades de los LLMs para diversas aplicaciones, desde la automatización de tareas hasta la asistencia creativa y la generación de contenido.

Configuración de los grandes modelos de lenguaje (LLM Settings)

Al trabajar con *prompts*, interactuamos con el LLM a través de una API o directamente. Podemos configurar algunos parámetros para obtener diferentes resultados a partir de nuestros *prompts*.

Temperatura: En resumen, cuanto más baja sea la temperatura, más deterministas serán los resultados en el sentido de que siempre se elige el siguiente token más probable. Aumentar la temperatura podría conducir a más aleatoriedad, lo que fomenta resultados más diversos o creativos. Esencialmente estás aumentando los pesos de los otros tokens posibles. En términos de aplicación, es posible que quieras usar un valor de temperatura más bajo para tareas como preguntas y respuestas basadas en hechos para fomentar respuestas más factuales y concisas. Para la generación de poemas u otras tareas creativas, podría ser beneficioso aumentar el valor de la temperatura.



<https://txt.cohere.com/llm-parameters-best-outputs-language-ai/>

Consideremos la frase "El cielo es". Cuando lees eso, probablemente piensas que la próxima palabra será "azul" o "el límite". También es improbable que la próxima palabra sea "agua" o "pegajoso". Inconscientemente hemos creado predicciones para palabras que pueden seguir y hemos determinado que "azul" es más probable mientras que "pegajoso" es menos probable.

Y eso es esencialmente lo que hace un modelo generativo de texto. Tiene probabilidades para todas las diferentes palabras que podrían seguir y luego selecciona la próxima palabra para producir. La configuración de Temperatura le indica cuáles de estas palabras puede usar.

Una Temperatura de 0 hace que el modelo sea determinista. Limita al modelo a usar la palabra con la mayor probabilidad. Podemos ejecutarlo una y otra vez y obtener el mismo resultado. A medida que aumentamos la Temperatura, el límite se suaviza, permitiéndole usar palabras con probabilidades más bajas, y podría generar "pegajoso" si lo ejecutamos suficientes veces.

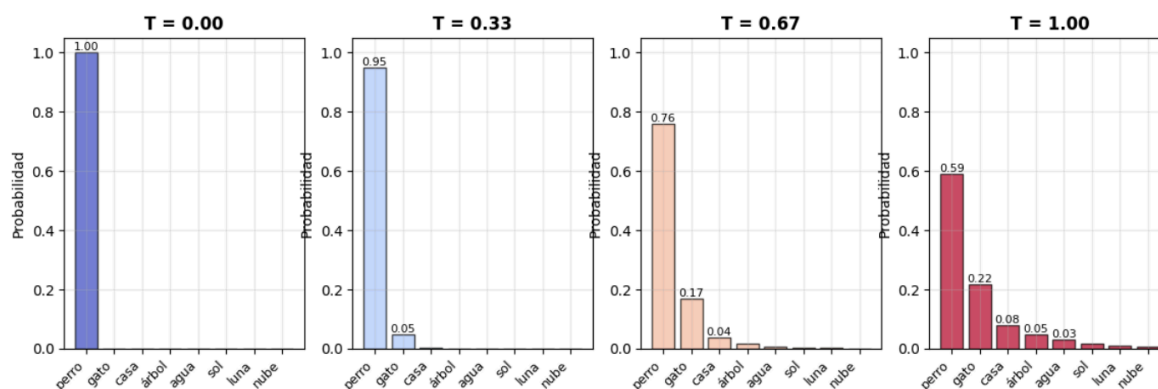
Cuando generamos texto desde un *prompt* "El cielo es" a una Temperatura media y baja, podemos ver resultados como por ejemplo:

El cielo es el límite

El cielo es azul

Y en la configuración más alta, entramos en el reino de la fantasía:

“El cielo es complicado, el agua tranquila, y es un camino inimaginablemente largo antes de que los delfines decidan renunciar a su búsqueda vertical”.



Distribución de probabilidades, según el valor de temperatura utilizado

Google Colab

https://colab.research.google.com/drive/1H7DRphqzTaFQJME5eM_Y6Uj4Kbiq3CDV?usp=sharing

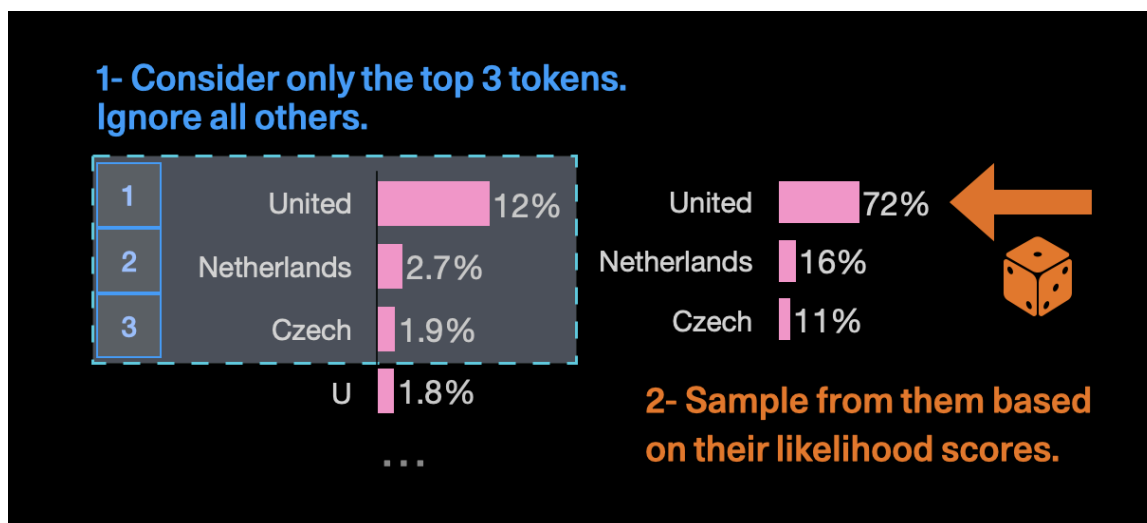


Top_p y Top_k:

Además de la Temperatura, Top-k y Top-p son las otras dos formas de elegir el token de salida.

Top-k le indica al modelo que elija el próximo token de los 'k' tokens principales en su lista, ordenados por probabilidad.

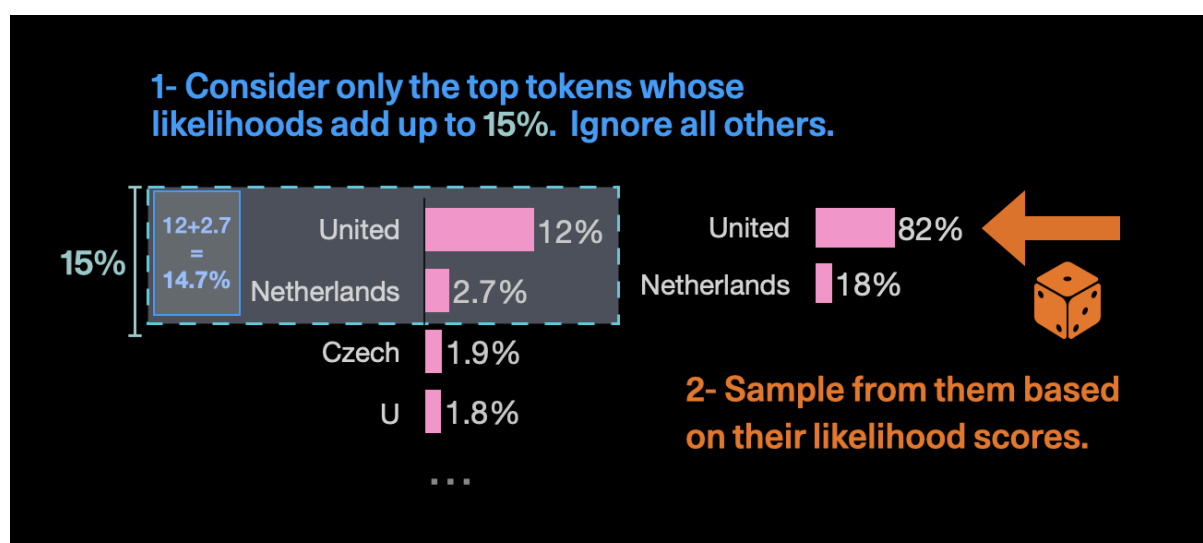
Consideremos la frase de entrada: "The name of that country is the" ("El nombre de ese país es el"). El próximo token podría ser "United", "Netherlands", "Czech", y así sucesivamente, con diferentes probabilidades. Puede haber docenas de salidas potenciales con probabilidades decrecientes, pero si establecemos k como 3, le estás diciendo al modelo que solo elija entre las 3 opciones principales.



Entonces, si ejecutamos el mismo prompt varias veces, obtendremos "United" muy a menudo, y de vez en cuando "Netherlands" o "Czech", pero nada más.

Si establecemos k en 1, el modelo solo elegirá el token superior ("United", en este caso).

Top-p es similar, pero elige entre los tokens superiores basándose en la suma de sus probabilidades. Entonces, para el ejemplo anterior, si establecemos top_p en 0.15, solo elegirá entre "United" y "Netherlands" ya que sus probabilidades suman el 14.7%.



A diferencia de Top-k, **Top-p** es más dinámico y **se usa a menudo para excluir salidas con probabilidades más bajas**. Entonces, si establecemos top_p en 0.75, excluimos el 25% inferior de las salidas probables.

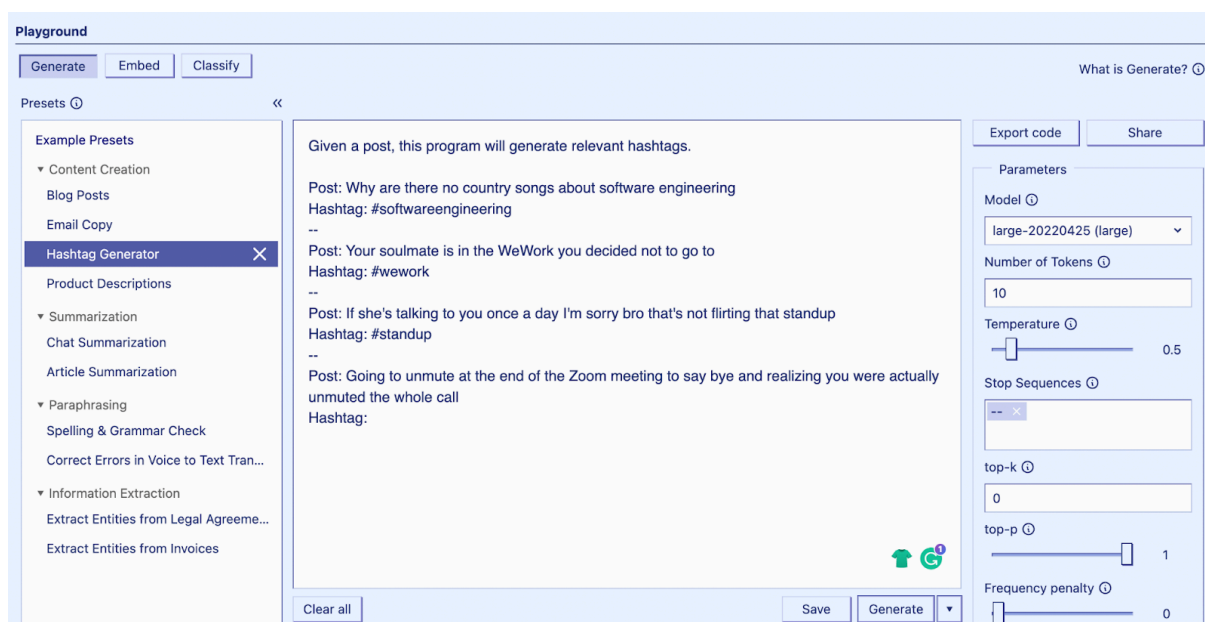
Max Length (Máxima longitud): Podemos gestionar el número de tokens que el modelo genera ajustando la 'máxima longitud'. Especificar una longitud máxima nos ayuda a prevenir respuestas largas o irrelevantes y a controlar costos.

Stop Sequences (Secuencias de Parada):

Una secuencia de parada es una cadena que le indica al modelo que deje de generar más contenido. Es otra forma de controlar cuán largo queremos el resultado.

Entonces, por ejemplo, si le damos al modelo el prompt "El cielo es" y escribo un punto (.) como secuencia de parada, el modelo deja de generar texto una vez que alcanza el final de la primera oración, incluso si el límite de tokens es mucho mayor.

Esto combina bien con prompts donde incluimos un par de ejemplos. Digamos que queremos generar texto siguiendo un cierto patrón, agregamos una cierta cadena a los ejemplos y luego usamos esa cadena como secuencia de parada.



Frequency Penalty (Penalización de Frecuencia): La 'penalización de frecuencia' aplica una penalización al siguiente token proporcional a cuántas veces ese token ya apareció en la respuesta y la indicación. Cuanto mayor sea la penalización de frecuencia, menos probable será que una palabra aparezca

de nuevo. Esta configuración reduce la repetición de palabras en la respuesta del modelo al dar a los tokens que aparecen más una penalización mayor.

Presence Penalty (Penalización de Presencia): La 'penalización de presencia' también aplica una penalización a tokens repetidos pero, a diferencia de la penalización de frecuencia, la penalización es la misma para todos los tokens repetidos. Un token que aparece dos veces y un token que aparece 10 veces son penalizados igual. Esta configuración evita que el modelo repita frases demasiado a menudo en su respuesta. Si queremos que el modelo genere texto diverso o creativo, es posible que queramos usar una penalización de presencia más alta. O, si necesitamos que el modelo se mantenga enfocado, intenta usar una penalización de presencia más baja.

Similar a la temperatura y *top_p*, la recomendación general es alterar la penalización de frecuencia o presencia, no ambas.

Antes de comenzar con algunos ejemplos básicos, tengamos en cuenta que nuestros resultados pueden variar dependiendo de la versión de LLM que usemos.

Nociones básicas de prompts

Podemos lograr mucho con *prompts* simples, pero la calidad de los resultados depende de cuánta información le proporcionamos y de lo bien elaborada que esté. Un *prompt* puede contener información como la instrucción o pregunta que estamos pasando al modelo, e incluir otros detalles como contexto, entradas o ejemplos. Podemos usar estos elementos para instruir mejor al modelo y, como resultado, obtener mejores resultados.

Comencemos repasando un ejemplo básico de un *prompt* simple:

Prompt

```
El cielo es
```

Output:

```
El cielo es una región de la atmósfera terrestre que se encuentra sobre nosotros y que generalmente se ve de color azul durante el día debido a la dispersión de la luz solar
```

por las partículas en la atmósfera.

Como podemos ver, el modelo de lenguaje produce una continuación de cadenas que tiene sentido dado el contexto "El cielo es". La salida puede ser inesperada o lejana de la tarea que queremos lograr.

Este ejemplo básico también destaca la necesidad de proporcionar más contexto o instrucciones sobre lo que específicamente queremos lograr.

Intentemos mejorarlo un poco:

Prompt:

Completa con una sola frase: El cielo está

Output:

El cielo está despejado y azul en un día soleado de verano.

¿Es eso mejor? Bueno, le dijimos al modelo que completara la oración, así que el resultado parece mucho mejor ya que sigue exactamente lo que le dijiste que hiciera ("completa la oración"). Este enfoque de diseñar *prompts* óptimos para instruir al modelo para realizar una tarea, es lo que se conoce como *ingeniería de prompts*.

El ejemplo anterior es una ilustración básica de lo que es posible con los LLM hoy en día. Los LLM de hoy son capaces de realizar todo tipo de tareas avanzadas que van desde la resumir textos hasta el razonamiento matemático y la generación de código.

Formateo de Prompts

Hemos probado un *prompt* muy simple anteriormente. Un *prompt* estándar tiene el siguiente formato:

<Pregunta>?

ó

<Instrucción>

Podemos formatear esto en un formato de preguntas y respuestas (QA), que es estándar en muchos conjuntos de datos de QA, de la siguiente manera:

Q: <Pregunta>
A:

Cuando hacemos indicaciones como la anterior, también se le llama indicación zero-shot (cero disparos), es decir, estamos solicitando directamente al modelo una respuesta sin ningún ejemplo o demostración sobre la tarea que queremos que logre. Algunos modelos de lenguaje a gran escala tienen la capacidad de realizar indicaciones zero-shot, pero depende de la complejidad y conocimiento de la tarea en cuestión.

Dado el formato estándar anterior, una técnica popular y efectiva para hacer indicaciones se conoce como prompt few-shot (de pocos disparos), donde proporcionamos ejemplos (es decir, demostraciones). Podemos formatear los *prompts* few-shot de la siguiente manera:

<Question>
<Answer>
<Question>
<Answer>
<Question>
<Answer>
<Question>

La versión de formato QA se vería así:

Q: <Question>
A: <Answer>
Q: <Question>
A: <Answer>
Q: <Question>
A: <Answer>
Q: <Question>
A:

Tengamos en cuenta que no es necesario usar el formato QA. El formato del prompt depende de la tarea en cuestión. Por ejemplo, podemos realizar una

tarea de clasificación simple y dar ejemplos que demuestren la tarea de la siguiente manera:

Prompt:

```
Texto: ¡Esto es malo!  
Etiqueta: Positivo  
Texto: ¡Vaya, esa película fue genial!  
Etiqueta: Negativo  
Texto: ¡Qué espectáculo tan horrible!  
Etiqueta: Positivo  
Texto: ¡Esto es increíble!  
Etiqueta:
```

Salida:

Negativo



Los *prompts few-shot* permiten el aprendizaje en contexto (in-context learning), que es la capacidad de los modelos de lenguaje para aprender tareas dadas unas pocas demostraciones.

Elementos de los Prompts

A medida que cubrimos más y más ejemplos y aplicaciones con la ingeniería de prompts, notaremos que ciertos elementos constituyen un prompt.

Un prompt contiene cualquiera de los siguientes elementos:

Instrucción - una tarea específica o instrucción que quieres que el modelo realice.

Contexto - información externa o contexto adicional que puede dirigir al modelo hacia respuestas mejores.

Datos de Entrada - la entrada o pregunta para la cual estamos interesados en encontrar una respuesta.

Indicador de Salida - el tipo o formato de la salida.

No necesitas todos los cuatro elementos para una indicación y el formato depende de la tarea en cuestión. Tocaremos ejemplos más concretos en guías próximas.

Diseño de Prompts

Aquí hay algunos consejos a tener en cuenta mientras diseñas tus indicaciones:

Comenzar Simple

A medida que comiences con el diseño de indicaciones, debes tener en cuenta que realmente es un proceso iterativo que requiere mucha experimentación para obtener resultados óptimos. Usar un entorno de pruebas simple de OpenAI o Cohere es un buen punto de partida.

Podemos comenzar con *prompts* simples y seguir agregando más elementos y contexto a medida que apuntamos a obtener mejores resultados. Cuando tengamos una tarea grande que involucre muchas sub-tareas diferentes, podemos intentar desglosar la tarea en sub-tareas más simples y seguir construyendo a medida que obtenemos mejores resultados. Esto evita añadir demasiada complejidad al proceso de diseño de *prompt* al principio.

Instrucciones

Podemos diseñar *prompts* efectivos para varias tareas simples usando comandos para instruir al modelo sobre lo que quieres lograr, como "Escribe", "Clasifica", "Resume", "Traduce", "Ordena", etc.

Tengamos en cuenta que también necesitamos experimentar mucho para ver qué funciona mejor. Podemos probar diferentes instrucciones con diferentes palabras clave, contextos y datos y ver qué funciona mejor para cada caso de uso y tarea en particular. Por lo general, cuanto más específico y relevante sea el contexto para la tarea que estamos tratando de realizar, mejor. Tocaremos la importancia del muestreo y la adición de más contexto más adelante.

Algunas recomendaciones indican conveniente colocar las instrucciones al principio del prompt. Otra recomendación es usar algún separador claro como "###" para separar la instrucción y el contexto.

Por ejemplo:

Prompt:

```
### Instrucción ###  
Traduce el texto a continuación al español.
```

Texto: "hello!"

Salida:

¡Hola!

Especificidad

Debemos ser específicos sobre la instrucción y la tarea que queremos que el modelo realice. Cuanto más descriptiva y detallada sea la indicación, mejores serán los resultados. Esto es particularmente importante cuando tenemos un resultado deseado o un estilo de generación que estás buscando. No hay tokens o palabras clave específicas que lleven a mejores resultados. Es más importante tener un buen formato y una indicación descriptiva. De hecho, proporcionar ejemplos en el *prompts* es muy efectivo para obtener la salida deseada en formatos específicos.

Al diseñar *prompts*, también debemos tener en cuenta la longitud del *prompt*, ya que hay limitaciones respecto a cuán larga puede ser (cantidad de tokens). Incluir demasiados detalles innecesarios no es necesariamente un buen enfoque. Los detalles deben ser relevantes y contribuir a la tarea en cuestión. Esto es algo con lo que debemos experimentar mucho. Es importante la experimentación e iteración para optimizar los *prompts* para nuestras aplicaciones.

Como ejemplo, intentemos un *prompt* simple para extraer información específica de un texto.

Prompt:

```
Extraer cantidades de conjuntos de datos climáticos realizados en el siguiente texto.  
Formato deseado: <cantidad_de_lugares>,<ambito_de_examen><crLf>  
Entrada:  
En el análisis se examinaron las temperaturas medias diarias y las olas de calor e incluyó datos de 175 países, 154 estados o provincias y 920 grandes ciudades. El informe se conoció a 20 días de que se celebre la cumbre de cambio climático en Dubai (COP28) en donde los líderes mundiales debatirán los avances en la lucha contra el calentamiento global.
```

Salida:

```
175,países
154,estados/provincias
920,ciudades
```

En el ejemplo, usamos los caracteres `<crLf>` (Retorno de carro y nueva línea), para generar una salida en diferentes renglones.

Evitar la imprecisión

Dado los consejos anteriores sobre ser detallado y mejorar el formato, es fácil caer en la trampa de querer ser demasiado astuto con las indicaciones y potencialmente crear descripciones imprecisas. A menudo es mejor ser específico y directo. La analogía aquí es muy similar a la comunicación efectiva: cuanto más directo, más eficazmente se transmite el mensaje.

Por ejemplo, podríamos estar interesados en aprender el concepto de ingeniería de *prompts*. Podemos intentar algo como:

```
Explicar el concepto de prompt engineering. Realizar una explicación breve, solo unas pocas oraciones, y no seas demasiado descriptivo.
```

No está claro en el *prompt* anterior cuántas frases usar y qué estilo. Aún podríamos obtener respuestas algo buenas con los *prompts* anteriores, pero un mejor *prompt* sería uno que sea más específico, conciso y al grano. Algo como:

```
Utilice 2 o 3 oraciones para explicar el concepto de prompt engineering a un estudiante de secundaria.
```

¿Hacer o no hacer?

Otro consejo común al diseñar *prompts*, es evitar decir lo que no se debe hacer, sino decir lo que se debe hacer en su lugar. Esto fomenta más especificidad y se centra en los detalles que conducen a buenas respuestas del modelo.

Aquí hay un ejemplo de un chatbot de recomendación de películas que falla exactamente en lo que no quiero que haga debido a cómo escribimos la instrucción, enfocándonos en lo que no debe hacer.

Prompt:

El siguiente es un agente que recomienda películas a un cliente. NO CONSULTAR POR INTERESES. NO PIDA INFORMACIÓN PERSONAL.

Cliente: Por favor recomiende una película basada en mis intereses.
Agente:

Salida:

Claro, puedo recomendarte una película según tus intereses. ¿Qué tipo de película te gustaría ver? ¿Prefieres acción, comedia, romance o algo más?

Acá hay un prompt mejorado:

Prompt:

El siguiente es un agente que recomienda películas a un cliente. El agente es responsable de recomendar una película de las principales tendencias mundiales. Debe abstenerse de preguntar a los usuarios sus preferencias y evitar solicitar información personal. Si el agente no tiene una película para recomendar, debería responder "Lo siento, no pude encontrar una película para recomendar hoy".

Cliente: recomiende una película según mis intereses.
Agente:

Salida:

Lo siento, no tengo ninguna información sobre tus intereses. Sin embargo, aquí hay una lista de las películas de mayor tendencia mundial en este momento: [lista de películas]. ¡Espero que encuentres algo que te guste!

Algunos de los ejemplos anteriores fueron tomados del artículo "[Mejores prácticas para la ingeniería de indicaciones con la API de OpenAI](#)".

Ejemplos de Prompts

La sección proporciona ejemplos sobre cómo utilizar *prompts* para lograr diferentes tareas e introduce conceptos clave en el camino. A menudo, la mejor manera de aprender conceptos es a través de ejemplos. Los ejemplos a continuación ilustran cómo se pueden usar indicaciones bien elaboradas para realizar diferentes tipos de tareas:

Resumen de textos

Una de las tareas estándar en la generación de lenguaje natural es el resumen de texto. La síntesis de texto puede incluir muchos enfoques y dominios diferentes. De hecho, una de las aplicaciones más prometedoras de los modelos de lenguaje es la capacidad de resumir artículos y conceptos en resúmenes rápidos y fáciles de leer. Probemos una tarea básica de resumen utilizando indicaciones.

Digamos que estás interesado en aprender sobre antibióticos, podrías intentar una indicación como esta:

Prompt:

Explique los antibióticos.
A:

Salida:

Los antibióticos son un tipo de medicamento que se usa para tratar infecciones bacterianas. Actúan matando las bacterias o impidiendo que se reproduzcan, lo que permite que el sistema inmunológico del cuerpo combata la infección. Los antibióticos generalmente se toman por vía oral en forma de píldoras, cápsulas o soluciones líquidas o, a veces, se administran por vía intravenosa. No son eficaces contra las infecciones virales y su uso inadecuado puede provocar resistencia a los antibióticos.

El "A:" es un formato de indicación explícito que se utiliza en las respuestas a preguntas. Aquí se utilizó para decirle al modelo que se espera una respuesta a continuación. En este ejemplo, no está claro cómo esto es útil frente a no usarlo, pero lo dejaremos así para ejemplos posteriores. Supongamos simplemente que esta es demasiada información y quieres resumirla aún más. De hecho, podemos instruir al modelo para que resuma en una sola oración de la siguiente manera:

Prompt:

Los antibióticos son un tipo de medicamento que se usa para tratar infecciones bacterianas. Actúan matando las bacterias o impidiendo que se reproduzcan, lo que permite que el sistema inmunológico del cuerpo combata la infección. Los antibióticos generalmente se toman por vía oral en forma de píldoras, cápsulas o soluciones líquidas o, a veces, se administran por vía intravenosa. No son eficaces contra las infecciones virales y su uso inadecuado puede provocar resistencia a los antibióticos.

Resume lo anterior en una sola oración:

Salida:

Los antibióticos son medicamentos que se usan para tratar infecciones bacterianas, ya sea matando las bacterias o impidiendo que se reproduzcan, pero no son eficaces contra los virus y el uso excesivo puede provocar resistencia a los antibióticos.

Sin prestar demasiada atención a la precisión de la salida anterior, el modelo intentó resumir el párrafo en una sola frase.

Extracción de información (Information Extraction)

Mientras que los modelos de lenguaje están entrenados para realizar la generación de lenguaje natural y tareas relacionadas, también son muy capaces de realizar clasificación y una variedad de otras tareas de procesamiento de lenguaje natural (NLP).

Aquí hay un ejemplo de una indicación que extrae información de un párrafo dado.

Prompt:

Las declaraciones de contribución de los autores y los agradecimientos en los artículos de investigación deben indicar clara y específicamente si los autores utilizaron tecnologías de inteligencia artificial como ChatGPT en la preparación de su manuscrito y análisis, y en qué medida. También deben indicar qué LLM se utilizaron. Esto alertará a los editores y revisores para que examinen los manuscritos con más atención en busca de posibles sesgos, inexactitudes y acreditación inadecuada de las fuentes. Del mismo modo, las revistas científicas deben ser transparentes sobre el uso de los LLM, por ejemplo, al seleccionar los manuscritos enviados.

Mencione el producto basado en modelos de lenguaje grande mencionado en el párrafo anterior:

Salida:

El producto basado en modelos de lenguaje grande mencionado en el párrafo anterior es ChatGPT.

Hay muchas maneras en las que podemos mejorar los resultados anteriores, pero esto ya es muy útil. A estas alturas nos damos cuenta que podemos pedirle al modelo que realice diferentes tareas simplemente instruyéndolo sobre qué hacer. Esa es una capacidad poderosa que los desarrolladores de

productos de inteligencia artificial pueden utilizarse para construir productos y experiencias poderosas.

Fuente del párrafo: [ChatGPT: cinco prioridades para la investigación \(se abre en una nueva pestaña\)](#).

Extracción de información estructurada:

Supongamos que queremos extraer información de un texto, pero luego queremos aplicar herramientas de programación para procesar la salida de forma automática. Podríamos trabajar el prompt de manera tal que seamos mucho más específicos en cuanto al formato de salida. Veamos el siguiente ejemplo, para la extracción de tríadas:

Actúa como un experto en extracción de conocimiento y análisis semántico. Tu tarea es extraer las principales relaciones semánticas en forma de tríadas (Sujeto - Relación - Objeto) del texto proporcionado.

****Instrucciones detalladas:****

1. ****Identifica las entidades principales:**** Estas serán los Sujetos y Objetos de tus tríadas. Pueden ser personas, organizaciones, conceptos, procesos, etc.
2. ****Identifica las relaciones:**** Estas son las acciones, estados o propiedades que conectan un Sujeto con un Objeto. Generalmente son verbos o frases verbales.
3. ****Formato de la tríada (Conceptual):**** Cada tríada debe seguir la estructura conceptual Sujeto - Relación - Objeto.
4. ****Precisión y Concisión:****
 - * Sé lo más específico posible al identificar cada componente de la tríada.
 - * Evita información superflua. La relación debe ser el núcleo del vínculo.
 - * Si una oración contiene múltiples ideas conectadas, puedes extraer múltiples tríadas.
5. ****Contexto:**** Considera el contexto de la frase para desambiguar y extraer la relación más significativa.
6. ****Formato de Salida Obligatorio:**** Devuelve cada tríada extraída en una ****nueva línea****, siguiendo un formato similar a una declaración de relación en Cypher. Representa cada tríada como:
``("ENTIDAD_SUJETO")-[:TIPO_RELACION]->("ENTIDAD_OBJETO")``
 - * ``ENTIDAD_SUJETO`` y ``ENTIDAD_OBJETO`` deben ser los nombres literales de las entidades extraídas del texto, tal como aparecen o con una mínima normalización si es necesario para la claridad (ej. resolver pronombres), y ****deben estar entre comillas dobles****.
 - * ``TIPO_RELACION`` debe ser una representación concisa y descriptiva de la relación, en ****MAYÚSCULAS****, usando ****guiones bajos (`\_`)**** para separar palabras si la relación consta de varias. Debe estar encerrada entre ``[:`` y ``]``.
 - * Si no se encuentran tríadas claras o relevantes en el texto, devuelve la cadena: ``No se encontraron tríadas.``

****Ejemplo de formato de salida para una tríada:****

Si el texto fuera "Ana es amiga de Pedro y Pedro trabaja en Consultora X.", la salida esperada sería:

```
`("Ana")-[:ES_AMIGA_DE]->("Pedro")`
`("Pedro")-[:TRABAJA_EN]->("Consultora X")`
```

****Texto a procesar:****

"El aire en la mansión Blackwood se podía cortar con un cuchillo tras el inesperado anuncio de Lord Alistair. Su sobrina, la joven y aparentemente ingenua Clara, observaba con disimulada atención la reacción de todos, especialmente la de su primo lejano, Julian, con quien se rumoreaba compartía algo más que un lazo familiar esporádico y alguna que otra inversión fallida heredada del abuelo de ambos. Beatrice, la matriarca y hermana de Alistair, siempre había desconfiado de las intenciones de Julian hacia la fortuna familiar, una fortuna que ella misma esperaba administrar a través de su hijo, el indolente pero astuto Edward, quien a su vez mantenía una secreta correspondencia con una tal "Seraphina", figura misteriosa que nadie en la familia admitía conocer pero cuyo nombre había sido susurrado en conexión con algunos negocios turbios del difunto primer esposo de Beatrice, el Coronel Montgomery.

El detective Harding, llamado para investigar no un crimen aún, sino las veladas amenazas que Lord Alistair decía recibir, sentía que cada conversación era un campo minado. Por ejemplo, la lealtad de Thomas, el mayordomo de toda la vida, hacia Lord Alistair parecía inquebrantable, pero Harding había interceptado una nota donde Thomas pedía consejo a su hermana, Penélope, quien casualmente trabajaba como ama de llaves para el principal rival de negocios de Alistair, el Sr. Croft. Croft, por su parte, negaba cualquier animosidad, aunque su hija, la ambiciosa Eleonor Croft, no ocultaba su desdén por los Blackwood, en especial por Edward, a quien consideraba un obstáculo para sus propios planes de expansión, planes que, curiosamente, había discutido en detalle con Julian en un discreto café la semana anterior. Clara, mientras tanto, había sido vista entregando un pequeño paquete a un desconocido cerca del muelle, un hombre que Harding reconoció vagamente como un antiguo asociado del Coronel Montgomery. La red de conexiones se sentía densa y opaca, y el detective sabía que desentrañarla sería la clave."

La respuesta del LLM, podría ser la siguiente:

```
("Lord Alistair")-[:HIZO_UN_ANUNCIO_INESPERADO]->("aire en la mansión Blackwood")
("Clara")-[:ES_SOBRINA_DE]->("Lord Alistair")
("Clara")-[:OBSERVABA_REACCION_DE]->("todos")
("Clara")-[:OBSERVABA_ESPECIALMENTE_REACCION_DE]->("Julian")
("Clara")-[:SE_RUMOREABA_COMPARTIA_ALGO_MAS_QUE_LAZO_FAMILIAR_CON]->("Julian")
("Clara")-[:SE_RUMOREABA_COMPARTIA_INVERSION_FALLIDA_CON]->("Julian")
("Inversión fallida")-[:FUE_HEREDADA_DE]->("abuelo de ambos (Clara y Julian)")
("Beatrice")-[:ES_MATRIARCA_DE_LA_FAMILIA]->("Blackwood")
("Beatrice")-[:ES_HERMANA_DE]->("Lord Alistair")
("Beatrice")-[:SIEMPRE_HABIA_DESCONFIADO_DE_INTENCIONES_DE]->("Julian")
("Julian")-[:TENIA_INTENCIONES_HACIA]->("fortuna familiar")
("Beatrice")-[:ESPERABA_ADMINISTRAR]->("fortuna familiar")
("Beatrice")-[:ESPERABA_ADMINISTRAR_A_TRAVES_DE]->("Edward")
("Edward")-[:ES_HIJO_DE]->("Beatrice")
("Edward")-[:MANTENIA_SECRETA_CORRESPONDENCIA_CON]->("Seraphina")
("Seraphina")-[:ES_FIGURA_MISTERIOSA_QUE_NADIE_ADMITE_CONOCER_EN]->("familia")
("Nombre de Seraphina")-[:HABIA_SIDO_SUSURRADO_EN_CONEXION_CON]->("negocios turbios del Coronel Montgomery")
("Coronel Montgomery")-[:FUE_DIFUNTO_PRIMER_ESPOSO_DE]->("Beatrice")
("Detective Harding")-[:FUE_LLAMADO_PARA_INVESTIGAR]->("veladas amenazas")
("Lord Alistair")-[:DECIA_RECIBIR]->("veladas amenazas")
("Lealtad de Thomas")-[:HACIA]->("Lord Alistair")
```

```
(
"Lealtad de Thomas")-[:PARECIA_INQUEBRANTABLE]->("Lord Alistair")
("Thomas")-[:ES_MAYORDOMO_DE_TODA_LA_VIDA_DE]->("Lord Alistair")
("Detective Harding")-[:HABIA_INTERCEPTADO_NOTA_DE]->("Thomas")
("Thomas")-[:PEDIA_CONSEJO_A]->("Penélope")
("Penélope")-[:ES_HERMANA_DE]->("Thomas")
("Penélope")-[:TRABAJABA_COMO_AMA_DE_LLAVES_PARA]->("Sr. Croft")
("Sr. Croft")-[:ES_PRINCIPAL_RIVAL_DE_NEGOCIOS_DE]->("Lord Alistair")
("Sr. Croft")-[:NEGABA]->("cualquier animosidad")
("Eleonor Croft")-[:ES_HIJA_DE]->("Sr. Croft")
("Eleonor Croft")-[:NO_OCULTABA_DESDEN_POR]->("los Blackwood")
("Eleonor Croft")-[:NO_OCULTABA_DESDEN_ESPECIAL_POR]->("Edward")
("Eleonor Croft")-[:CONSIDERABA_OBSTACULO_A]->("Edward")
("Edward")-[:ES_OBSTACULO_PARA_PLANES_DE_EXPANSION_DE]->("Eleonor Croft")
("Eleonor Croft")-[:HABIA_DISCUTIDO_PLANES_CON]->("Julian")
("Clara")-[:HABIA_SIDO_VISTA_ENTREGANDO_PAQUETE_A]->("un desconocido")
("Desconocido")-[:ESTABA_CERCA_DE]->("el muelle")
("Detective Harding")-[:RECONOCIO_VAGAMENTE_COMO_ANTIGUO_ASOCIADO_DEL_CORONEL_MONTGOME
RY_A]->("un desconocido")
("Un desconocido")-[:ES_ANTIGUO_ASOCIADO_DE]->("Coronel Montgomery")
)
```

Podríamos trabajar aún más el *prompt* para nuestro caso de uso, e incluso puede ser necesario aplicar filtros posteriores en nuestra aplicación, pero vemos la potencia que tienen los modelos para crear salidas estructuradas a partir de escribir la instrucción de una manera clara y asertiva, y especificando el resultado que esperamos obtener.

Responder preguntas (Question Answering)

Una de las mejores maneras de hacer que el modelo responda a respuestas específicas es mejorar el formato del *prompt*. Como se mencionó antes, un *prompt* puede combinar instrucciones, contexto, entrada y señales de salida para obtener resultados mejorados. Aunque estos componentes no son necesarios, se convierte en una buena práctica ya que cuanto más específico seamos con la instrucción, mejores resultados obtendremos. A continuación hay un ejemplo de cómo se vería esto siguiendo un *prompt* más estructurado.

Prompt:

Responda la pregunta basándose en el contexto siguiente. Mantenga la respuesta breve. Responda "No estoy seguro de la respuesta" si no está seguro de la respuesta.

Contexto: El teplizumab tiene sus raíces en una compañía farmacéutica de Nueva Jersey llamada Ortho Pharmaceutical. Allí, los científicos generaron una versión temprana del anticuerpo, denominada OKT3. Originaria de ratones, la molécula pudo unirse a la superficie de las células T y limitar su potencial de destrucción celular. En 1986, fue aprobado para ayudar a prevenir el rechazo de órganos después de trasplantes de riñón, lo que lo convirtió en el primer anticuerpo terapéutico permitido para uso humano.

Pregunta: ¿De qué se obtuvo originalmente OKT3?

Respuesta:

Salida:

OKT3 se obtuvo originalmente de ratones.

Contexto obtenido de: [Nature](#)(opens in a new tab).

Clasificación de texto (Text Classification)

Hasta ahora, hemos utilizado instrucciones simples para realizar una tarea. En casos más difíciles, proporcionar solo instrucciones no será suficiente. Aquí es donde necesitamos pensar más en el contexto y los diferentes elementos que podemos usar en un *prompt*. Otros elementos que podemos proporcionar son datos de entrada o ejemplos.

Intentemos demostrar esto proporcionando un ejemplo de clasificación de texto.

Prompt:

Clasifica el texto en neutro, negativo o positivo.

Texto: Creo que la comida estuvo bien.

Sentimiento:

Salida:

Neutro

Le dimos la instrucción de clasificar el texto y el modelo respondió con 'Neutro', que es correcto. No hay nada malo con esto, pero digamos que lo que realmente necesitamos es que el modelo dé la etiqueta en el formato exacto que deseamos. Entonces, en lugar de *Neutro*, queremos que devuelva *neutro*. ¿Cómo logramos esto? Hay diferentes maneras de hacerlo. Aquí nos importa la especificidad, por lo que cuanto más información podamos proporcionar al *prompt*, mejores serán los resultados. Podemos intentar proporcionar ejemplos para especificar el comportamiento correcto. Intentemos de nuevo:

Prompt:

Clasifica el texto en neutro, negativo o positivo.

Texto: Creo que las vacaciones están bien.

Sentimiento: neutro

Texto: Creo que la comida estuvo bien.

Sentimiento:

Salida:

neutro

¡Perfecto! Esta vez el modelo devolvió **neutro**, que es la etiqueta específica que buscábamos. Parece que el ejemplo proporcionado en el prompt ayudó al modelo a ser específico en su salida.

Para resaltar por qué a veces ser específico es importante, revisemos el ejemplo a continuación e identifiquemos el problema:

Prompt:

Clasifica el texto en nutral, negativo o positivo.

Texto: Creo que las vacaciones están bien.

Sentimiento:

Salida:

Neutral

¿Cuál es el problema aquí? Como pista, la etiqueta inventada "nutral" es completamente ignorada por el modelo. En cambio, el modelo emite "Neutral" ya que tiene cierto sesgo hacia esa etiqueta. Pero supongamos que lo que realmente quieres es "nutral". ¿Cómo arreglaríamos esto? ¿Quizá podamos intentar agregar descripciones a las etiquetas o agregar más ejemplos al *prompt*? Si no estamos seguros, discutiremos algunas ideas en las próximas secciones.

Conversación (Conversation)

Quizás una de las cosas más interesantes que podamos lograr con la ingeniería de *prompts* es instruir al sistema LLM sobre cómo comportarse, su intención y su identidad. Esto es particularmente útil cuando estamos construyendo sistemas conversacionales como chatbots de servicio al cliente.

Por ejemplo, pensemos un sistema conversacional que sea capaz de generar respuestas más técnicas y científicas a las preguntas. Observemos cómo le estamos diciendo explícitamente cómo comportarse a través de la instrucción. Esto a veces se conoce como *indicación de rol*.

Prompt:

La siguiente es una conversación con un asistente de investigación de IA. El tono del asistente debe ser técnico y científico.

Humano: Hola, ¿quién eres?

AI: ¡Saludo! Soy asistente de investigación de IA. ¿Cómo puedo ayudarte hoy?

Humano: ¿Puedes hablarme sobre la creación de agujeros negros?

AI:

Salida:

¡Seguro! Los agujeros negros son regiones del espacio-tiempo donde la fuerza gravitacional es tan fuerte que nada, ni siquiera la luz, puede escapar de ellos. Se crean cuando una estrella muy masiva muere y su núcleo colapsa sobre sí mismo, formando una singularidad de densidad infinita. La intensa gravedad de la singularidad atrae toda la materia y radiación a su alrededor, creando el agujero negro.

Nuestro asistente de investigación de IA suena un poco demasiado técnico, ¿verdad? Bien, cambiemos este comportamiento e instruyamos al sistema para que dé respuestas más accesibles.

Prompt:

La siguiente es una conversación con un asistente de investigación de IA. Las respuestas del asistente deben ser fáciles de entender incluso para los estudiantes de primaria.

Humano: Hola, ¿quién eres?

AI: ¡Saludo! Soy asistente de investigación de IA. ¿Cómo puedo ayudarte hoy?

Humano: ¿Puedes hablarme sobre la creación de agujeros negros?

AI:

Salida:

```
¡Seguro! Los agujeros negros se crean cuando una estrella muy masiva se queda sin combustible y colapsa sobre sí misma. La gravedad de la estrella es tan fuerte que nada, ni siquiera la luz, puede escapar de su atracción. Esto crea un punto en el espacio-tiempo con una intensa fuerza gravitacional llamado agujero negro.
```

Code Generation

Una aplicación en la que los Modelos de Lenguaje a Gran Escala (LLMs) son bastante efectivos es la generación de código. Copilot es un gran ejemplo de esto. Hay un gran número de tareas de generación de código que podemos realizar con *prompts*. Veamos algunos ejemplos a continuación.

Primero, intentemos un programa simple que salude al usuario.

Prompt:

```
/*  
Programa que pregunta al usuario su nombre y dice "Hola"  
*/
```

Salida:

```
// Programa que pregunta al usuario su nombre y dice "Hola"  
const prompt = require('prompt-sync')();  
  
// Pregunta al usuario su nombre  
const name = prompt('Por favor, introduce tu nombre: ');  
  
// Saluda al usuario  
console.log(`Hola, ${name}!`);
```

Observemos que ni siquiera necesitamos especificar el lenguaje a utilizar.

Cambemos un poco de nivel. El ejemplo a continuación muestra cuán poderosos pueden ser los LLMs, con un poco más de esfuerzo en el diseño de los *prompts*.

Prompt:

```
"""
Table departments, columns = [DepartmentId, DepartmentName]
Table students, columns = [DepartmentId, StudentId, StudentName]
Crear una consulta MySQL para todos los estudiantes del Departamento de Informática.
"""
```

Salida:

```
SELECT StudentId, StudentName
FROM students
WHERE DepartmentId IN (SELECT DepartmentId FROM departments WHERE DepartmentName = 'De
partamento de Informática');
```

En este caso, proporcionamos datos sobre el esquema de la base de datos y le pedimos que generara una consulta MySQL válida.
